

LINQ Concat Method in C# with Examples

In this article, I will discuss the **LINQ Concat Method in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ Union Method in C#](#) with Examples. As part of this article, we will discuss the following pointers.

1. [What is the Concat Method in LINQ?](#)
2. [Why do we need to use the Concat Method?](#)
3. [Examples using both Query and Method Syntax.](#)
4. [What are the differences between Concat and union operators in Linq?](#)
5. [When should you use the LINQ Concat Method in C#?](#)

LINQ Concat Method in C#:

The LINQ Concat Method in C# is used to concatenate two sequences into one sequence of the same type. It appends the elements of the second sequence to the end of the first sequence. That means it concatenates two of the same types of sequences or collections and returns a new sequence or collection without removing the duplicate elements.

The Concat method is a part of the **System.Linq** namespace can be used with any type implementing **IEnumerable<T>** interface. Only one version is available for this method, whose signature is shown below.

```
public static IEnumerable<TSource> Concat<TSource>(this IEnumerable<TSource> first,
IEnumerable<TSource> second);
```

Key Points About The LINQ Concat Method:

- **Combining Two Sequences:** Concat takes two sequences (like arrays, lists, or any other enumerable collections) and combines them into one. Both sequences must be of the same type.
- **Order Preservation:** The elements of the first sequence appear first in the resulting sequence, followed by the elements of the second sequence. The relative order of elements within each sequence is preserved.
- **Deferred Execution:** Like most LINQ methods, Concat uses deferred execution. The concatenated sequence is not created immediately when you call Concat. Instead, it's created when you iterate over the sequence (e.g., using a foreach loop).
- **No Distinction Between Duplicates:** Concat does not check for duplicates. If either of the original sequences contains duplicates or an element is present in both sequences, these duplicates are also present in the concatenated sequence.

Example to Understand Concat Method with Primitive Data Types in C#:

Let us understand the LINQ Concat Method with Primitive Data Types in C# using both Method and Query Syntax. In the following example, we have created two integer collections, which will be our data sources, and then we concatenate the two collections into one collection using the Concat Method. Here, you can see both data collection data types are the same, i.e., `List<int>`; otherwise, we will get a compilation error. In Query Syntax, there is no such operator called concat, so we need to use mixed syntax, i.e., both the query and method syntax, to achieve the same.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> sequence1 = new List<int> { 1, 2, 3, 4 };
            List<int> sequence2 = new List<int> { 2, 4, 6, 8 };
            //Method Syntax
            var MS = sequence1.Concat(sequence2);
            //Query Syntax
            var QS = (from num in sequence1
                      select num)
                      .Concat(sequence2).ToList();
            foreach (var item in MS)
            {
                Console.Write(item + " ");
            }
            Console.ReadLine();
        }
    }
}

```

Output: 1 2 3 4 2 4 6 8

If you notice in the above output, then you will see that the duplicate elements, i.e., 2 and 4, are not removed. The Concat Method will throw an exception if any sequences are null. In the below example, the second sequence is null, and while performing the concatenate operation using the LINQ Concat Operator, it will throw an exception.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> sequence1 = new List<int> { 1, 2, 3, 4 };
            List<int> sequence2 = null;
            var result = sequence1.Concat(sequence2);
            foreach (var item in result)
            {
                Console.WriteLine(item);
            }
            Console.ReadLine();
        }
    }
}

```

```

    }
}

```

Now run the application, and you will get the following exception.



LINQ Concat Method with Complex Data Type in C#:

Let us understand how the LINQ Concat Method works with Complex types. Create a class file named **Student.cs** and copy and paste the following code.

```

namespace LINQDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
    }
}

```

This is a very simple student class with just two properties. Let's say we have the following two data sources.

```

List<Student> StudentCollection1 = new List<Student>()
{
    new Student {ID = 101, Name = "Preety" },
    new Student {ID = 102, Name = "Sambit" },
    new Student {ID = 105, Name = "Hina"},
    new Student {ID = 106, Name = "Anurag"},
};

List<Student> StudentCollection2 = new List<Student>()
{
    new Student {ID = 105, Name = "Hina"},
    new Student {ID = 106, Name = "Anurag"},
    new Student {ID = 107, Name = "Pranaya"},
    new Student {ID = 108, Name = "Santosh"},
};

```

Now, we need to Concatenate the above two data sources into a single data source without removing the duplicate elements. Here, you can see two students appeared in both data sources. To do so, we need to

use the LINQ Concat method. So, modify the Main method of the Program class as follows.

```
using System.Collections.Generic;
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> StudentCollection1 = new List<Student>()
            {
                new Student {ID = 101, Name = "Preety" },
                new Student {ID = 102, Name = "Sambit" },
                new Student {ID = 105, Name = "Hina"},
                new Student {ID = 106, Name = "Anurag"},
            };
            List<Student> StudentCollection2 = new List<Student>()
            {
                new Student {ID = 105, Name = "Hina"},
                new Student {ID = 106, Name = "Anurag"},
                new Student {ID = 107, Name = "Pranaya"},
                new Student {ID = 108, Name = "Santosh"},
            };
            //Method Syntax
            var MS = StudentCollection1.Concat(StudentCollection2).ToList();
            //Query Syntax
            var QS = (from std in StudentCollection1
                      select std).Concat(StudentCollection2).ToList();
            foreach (var student in MS)
            {
                Console.WriteLine($" ID : {student.ID} Name : {student.Name}");
            }
            Console.ReadKey();
        }
    }
}
```

Run the application, and you will get the following output showing the duplicate elements.

```
ID : 101 Name : Preety
ID : 102 Name : Sambit
ID : 105 Name : Hina
ID : 106 Name : Anurag
ID : 105 Name : Hina
ID : 106 Name : Anurag
ID : 107 Name : Pranaya
ID : 108 Name : Santosh
```

When should you use the LINQ Concat Method in C#?

The LINQ Concat method in .NET is useful when you need to combine two sequences of the same type into a single sequence without removing the duplicates. Here are some scenarios when Concat is particularly useful:

- **Merging Collections:** When you have two lists, arrays, or other enumerable collections of the same type, you want to create a single sequence that includes elements from both collections in the order they appear.
- **Concatenating Results:** In cases where you have two sets of data retrieved from different sources (like two different database queries or a database query and a local collection), you want to process them as a single set.
- **Sequential Processing:** If you need to process elements from one collection followed by elements from another collection in a specific order.
- **Avoiding Duplicate Entries:** Unlike the Union method, which removes duplicates, Concat retains all original elements from both sequences, including duplicates. This is useful when preserving duplicates is necessary for your application logic.
- **Pagination Scenarios:** In some pagination scenarios, where you might want to append a new page of data to an existing sequence of items without modifying the original sequence.

What is the Difference Between LINQ Concat and Union Method in C#?

The Concat Method concatenates two sequences into one sequence without removing the duplicate elements. That means it simply returns the elements from the first sequence followed by the elements from the second sequence. On the other hand, the LINQ Union method is also used to concatenate two sequences into one sequence by removing duplicate elements. For a better understanding, please look at the following example, where we use both Concat and Union Methods on the same data sources.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Sources
            List<int> sequence1 = new List<int> { 1, 2, 3, 4 };
        }
    }
}
```

```

        List<int> sequence2 = new List<int> { 2, 4, 6, 8 };
        //Using Concat Method
        var ConcatResult = sequence1.Concat(sequence2);
        Console.WriteLine("Concat Method Result: ");
        foreach (var item in ConcatResult)
        {
            Console.Write(item + " ");
        }
        //Using Union Method
        var UnionResult = sequence1.Union(sequence2);
        Console.WriteLine("\nUnion Method Result: ");
        foreach (var item in UnionResult)
        {
            Console.Write(item + " ");
        }
        Console.ReadLine();
    }
}

```

Output:

```

Concat Method Result: 1 2 3 4 2 4 6 8
Union Method Result: 1 2 3 4 6 8

```

As you can see in the above output, the Concat method returns the duplicate elements, whereas the Union method removes the duplicate elements from the result set.

LINQ Concat Method:

- **Functionality:** The Concat method concatenates two sequences of the same type. Concat appends the second sequence to the first sequence.
- **Duplicates:** It does not remove duplicates. If an element appears in both sequences, it will appear twice in the result.
- **Order:** The order of elements is preserved as in the original sequences. If you have two sequences and you use Concat, the resulting sequence will contain all the elements from the first sequence, followed by all the elements from the second sequence, maintaining the order in which they appear.
- **Example:** If you concat two lists [1, 2, 3] and [3, 4, 5], the result will be [1, 2, 3, 3, 4, 5].
- **Comparer:** The Concat method does not use any equality comparer as it will not check equality.
- **Use Case:** Ideal when creating a single sequence that maintains the order and includes all elements, even if some are duplicates.

LINQ Union Method:

- **Functionality:** The Union method is also used to combine two sequences of the same type.
- **Duplicates:** It removes duplicate elements. It only includes distinct elements in the result, even if an element appears in both sequences. That means it returns a new sequence that contains the unique elements from both the original sequences.

- **Order:** There is no guarantee that the order of elements will be preserved.
- **Example:** Using the Union method on [1, 2, 3] and [3, 4, 5] will yield [1, 2, 3, 4, 5].
- **Comparer:** The Union method uses the default equality comparer for the type of elements in the sequences to determine whether two elements are equal.
- **Use Case:** Best used when you need a set that contains each distinct element from both sequences.

Key Differences Between Them:

- **Handling of Duplicates:** Concat keeps all duplicates from both sequences, whereas Union removes duplicates, keeping only distinct elements.
- **Order Preservation:** Concat maintains the order of elements as they appear in the original sequences, but Union does not guarantee the order of elements.
- **Performance:** Concat is generally faster than Union because it does not need to check for duplicates. Union can be slower, especially for larger collections, due to the overhead of checking and removing duplicates.

Choosing Between Concat and Union Method in C#:

- Use Concat when the order of elements is important, and you need to retain duplicates, such as when concatenating lists or arrays where each element's occurrence is significant.
- Use Union when you need a set union that removes duplicates, and the order of elements is not a priority.

In the next article, I will discuss the [Ordering Operators in LINQ](#) with Examples. In this article, I explain the **LINQ Concat Method in C#** and the **Difference Between Concat and Union Operator in C#** with Examples. I hope you understand the need and use of the LINQ Concat Method in C#.